# DIGISANCHIKA : REQUIRED API PREFERENCE

This document lists the API contracts required by the refined Home, My Documents, and Library UI.

## BASE URL AND AUTHENTICATION

All endpoints use the configured API base URL. Protected requests send:

Authorization: Bearer <access-token>

Core session endpoints:

- ? ‘POST /api/auth/login‘
- ? ‘POST /api/auth/logout‘
- ? ‘POST /api/auth/refresh‘
- ? ‘GET /api/auth/me‘

## HOME AND MY DOCUMENTS

- ? ‘GET /api/documents‘
- ? ‘GET /api/documents?folderId={folderId}‘
- ? ‘GET /api/documents/{documentId}‘
- ? ‘DELETE /api/documents/{documentId}‘
- ? ‘GET /api/folders‘
- ? ‘POST /api/folders‘
- ? ‘GET /api/folders/{folderId}‘
- ? ‘DELETE /api/folders/{folderId}‘

Folder contents should return { documents, folders, count } and preserve folderId, parentId, names, dates, and perm

## UPLOAD

- ? ‘POST /api/documents/upload-url‘
- ? ‘PUT /api/documents/upload/{documentId}‘
- ? ‘POST /api/documents/{documentId}/confirm-upload‘
- ? ‘POST /api/documents/bulk-upload-urls‘

Upload metadata must preserve folderId, fileName, fileSize, mimeType, and allowDownload.

LIBRARY AND PUBLISHING

Document publishing:

? ‘POST /api/documents/{documentId}/publish‘
? ‘POST /api/documents/{documentId}/unpublish‘

Folder publishing:

? ‘POST /api/folders/{folderId}/publish‘
? ‘POST /api/folders/{folderId}/unpublish‘

Library retrieval:

? ‘GET /api/library‘
? ‘GET /api/library/folders‘
? ‘GET /api/library/folders/{folderId}‘

The publish response should include isPublishedToLibrary, so the UI can switch between ?Publish to Library? and ?Un

DOWNLOADS

Document download:

? ‘GET /api/documents/{documentId}/download-url‘

Folder download:

? ‘GET /api/folders/{folderId}/download-url‘
? ‘GET /api/folders/{folderId}/download‘
? ‘GET /api/folders/{folderId}/download-manifest‘

Folder ZIP responses must preserve the root folder, nested folders, original file names, and download permissions.

SEARCH AND AI

? ‘POST /api/search/keyword‘
? ‘POST /api/search/semantic‘

? ‘GET /api/ai/status‘

Search payloads should support query, scope, and limit.

SHARING AND NOTIFICATIONS

Sharing:

? ‘GET /api/shares/users‘
? ‘GET /api/shares/shared-with-me‘
? ‘GET /api/shares/shared-by-me‘
? ‘GET|POST|PATCH /api/shares/documents/{documentId}‘
? ‘GET|POST|PATCH /api/shares/folders/{folderId}‘
? ‘POST /api/shares/documents/revoke/{shareId}‘
? ‘POST /api/shares/folders/revoke/{shareId}‘

Notifications:

? ‘GET /api/notifications‘
? ‘POST /api/notifications/{notificationId}/read‘
? ‘POST /api/notifications/read-all‘
? ‘DELETE /api/notifications/{notificationId}‘

MINIMUM PRODUCTION REQUIREMENTS

The backend must enforce authenticated ownership, folder hierarchy, persistent publish state, duplicate-publish pro